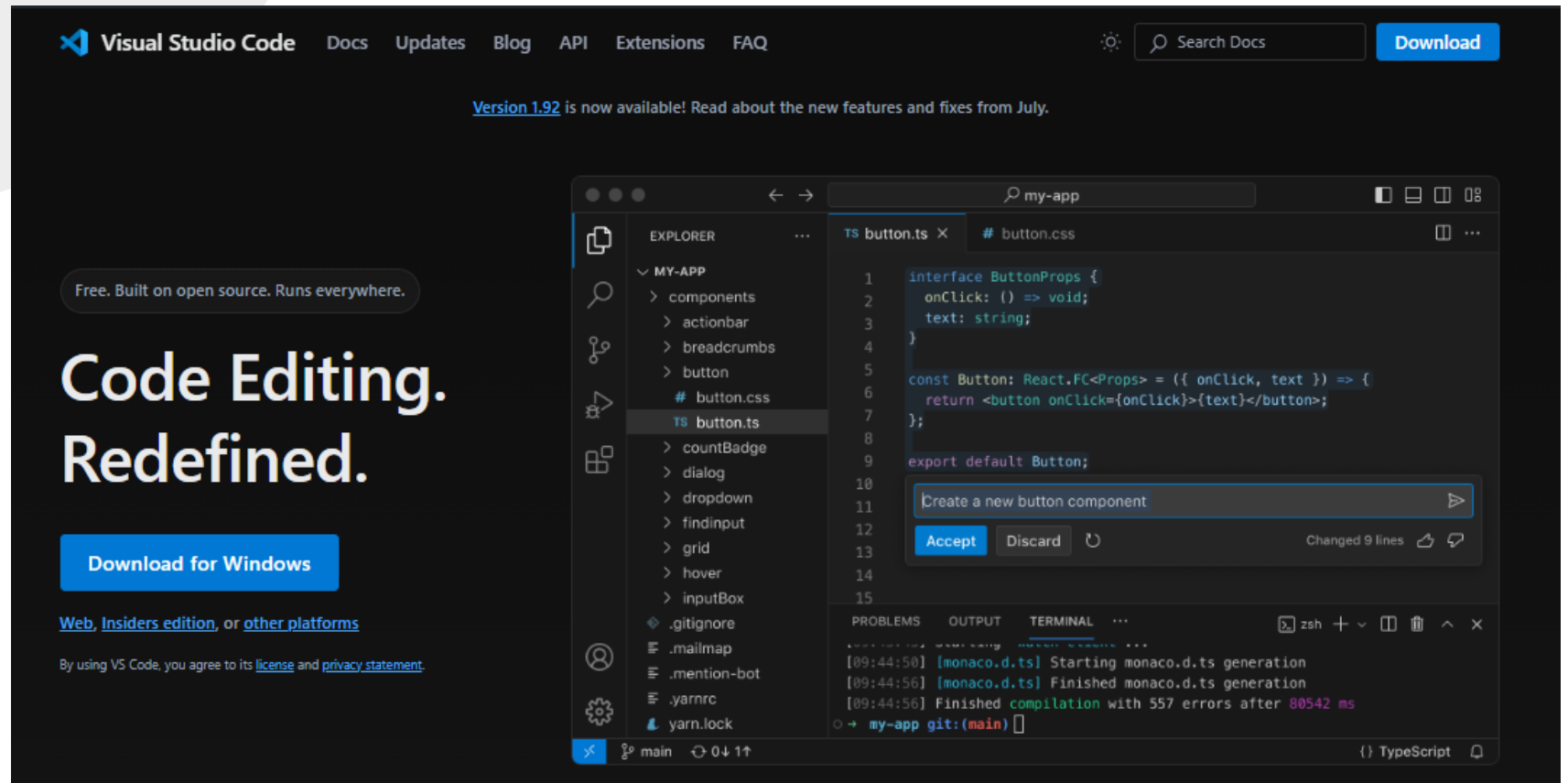


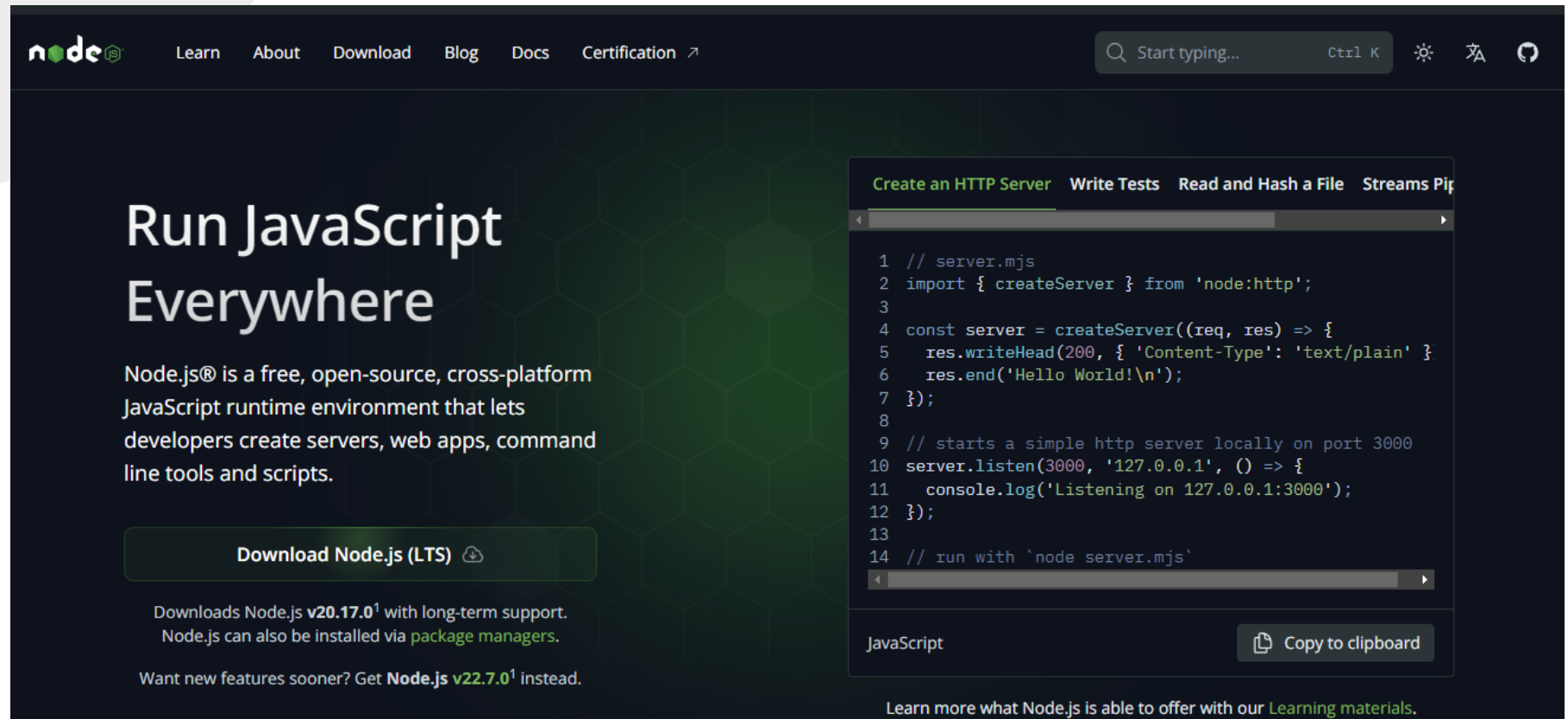


JAVASCRIPT FUNDAMENTALS

SETTING UP OUR CODE EDITOR



SETTING UP OUR CODE EDITOR



The screenshot shows the Node.js website with a dark theme. The main heading is "Run JavaScript Everywhere". Below it, a paragraph describes Node.js as a free, open-source, cross-platform JavaScript runtime environment. A "Download Node.js (LTS)" button is visible. A code editor overlay on the right shows a JavaScript snippet for creating an HTTP server. The code includes comments and uses the `createServer` function from `node:http`. A "Copy to clipboard" button is at the bottom of the code editor. The website's navigation bar includes links for Learn, About, Download, Blog, Docs, and Certification. A search bar and utility icons are also present.

node.js Learn About Download Blog Docs Certification ↗

Search Start typing... Ctrl K

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

[Download Node.js \(LTS\)](#)

Downloads Node.js **v20.17.0**¹ with long-term support. Node.js can also be installed via [package managers](#).

Want new features sooner? Get **Node.js v22.7.0**¹ instead.

Create an HTTP Server

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript [Copy to clipboard](#)

[Learn more what Node.js is able to offer with our Learning materials.](#)

The background features a large white semi-circle at the top. Below it, a green shape with a curved bottom edge spans the width of the slide. To the left of the green shape is a gray area, and to the right is a blue area.

What is JavaScript?

What is JavaScript?

JavaScript is a **programming language** used to create **dynamic** and **interactive** content on websites. It allows developers to build features like image sliders, pop-up messages, interactive forms, games, and more. Unlike HTML and CSS, which structure and style a webpage, JavaScript makes it functional and responsive to user actions.



Programming Language:

JavaScript is a set of instructions that you write to tell the computer what to do. Like a language used for communication between people, a programming language lets you communicate with computers.





Dynamic Content:

"Dynamic" means content that can change or move on the page. JavaScript allows you to create content that updates or reacts without needing to reload the page.

Interactive Content:

"Interactive" refers to content that responds to user actions. When you click a button and something happens (like a menu opening or an item being added to a shopping cart), that's JavaScript in action.





Features:

JavaScript is responsible for many of the features that make websites feel modern and user-friendly, like:

Image sliders: Slideshows of images that change automatically or when you click next.

Pop-up messages: Alerts or notifications that appear on the screen.

Interactive forms: Forms that check for errors or provide suggestions as you type.

Games: Simple games you can play in your web browser.

Primitive Data Types

Primitive Data Types

Primitive data types are the most basic kinds of data types. They are not objects and have no methods. There are seven primitive data types in JavaScript:

- String
- Number
- Boolean
- Null
- Undefined
- Symbol
- BigInt



Primitive Data Types

- **String:** String is a sequence of characters used to represent text.

"Hello, World!" and 'John Doe' are both strings. They can be enclosed in either single (') or double (") quotes.

=>(+, "\", .LENGTH, INDEX[])

- **Number:** A Number represents both integer and floating-point numbers.

30, 19.99, and -5 are all numbers in JavaScript. They can represent integers, decimals, and negative values.

=> (DO SOME MATH, MODULUS %)

Evaluate the following statements

1. `100 % 3`
2. `("blah" + "blah")[6]`
3. `"hello".length % "hi\\".length`

Primitive Data Types

- **Boolean:** A Boolean has only two possible values: true or false.

true and false are boolean values. They are often used in conditional statements to control the flow of code.

- **Null:** Null is a special value that represents "nothing" or "empty."

null is explicitly assigned to a variable to indicate that it has no value or is empty. It is often used to reset or clear variables.

- **Undefined**

Undefined means a variable has been declared, but no value has been assigned to it yet.

When a variable is declared but not initialized, it is automatically assigned the value undefined.

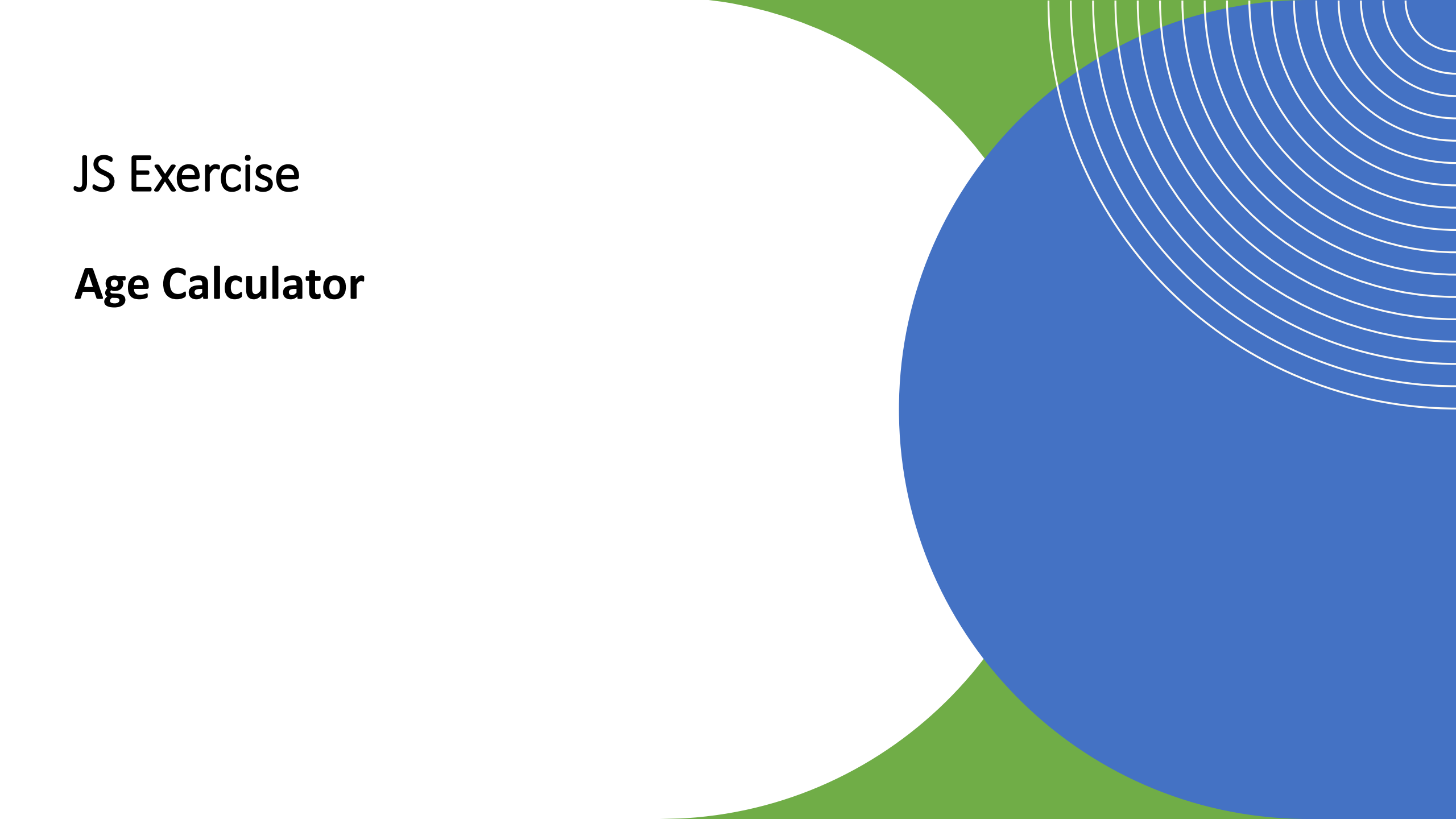
alert, prompt, console.log()

- **alert:** The alert() function is used to display a pop-up message to the user.
- **prompt:** The prompt() function is used to display a dialog box that prompts the user to input some text.
- **console.log():** The console.log() function is used to print messages to the browser's console, which is typically used for debugging purposes.

JS Exercise

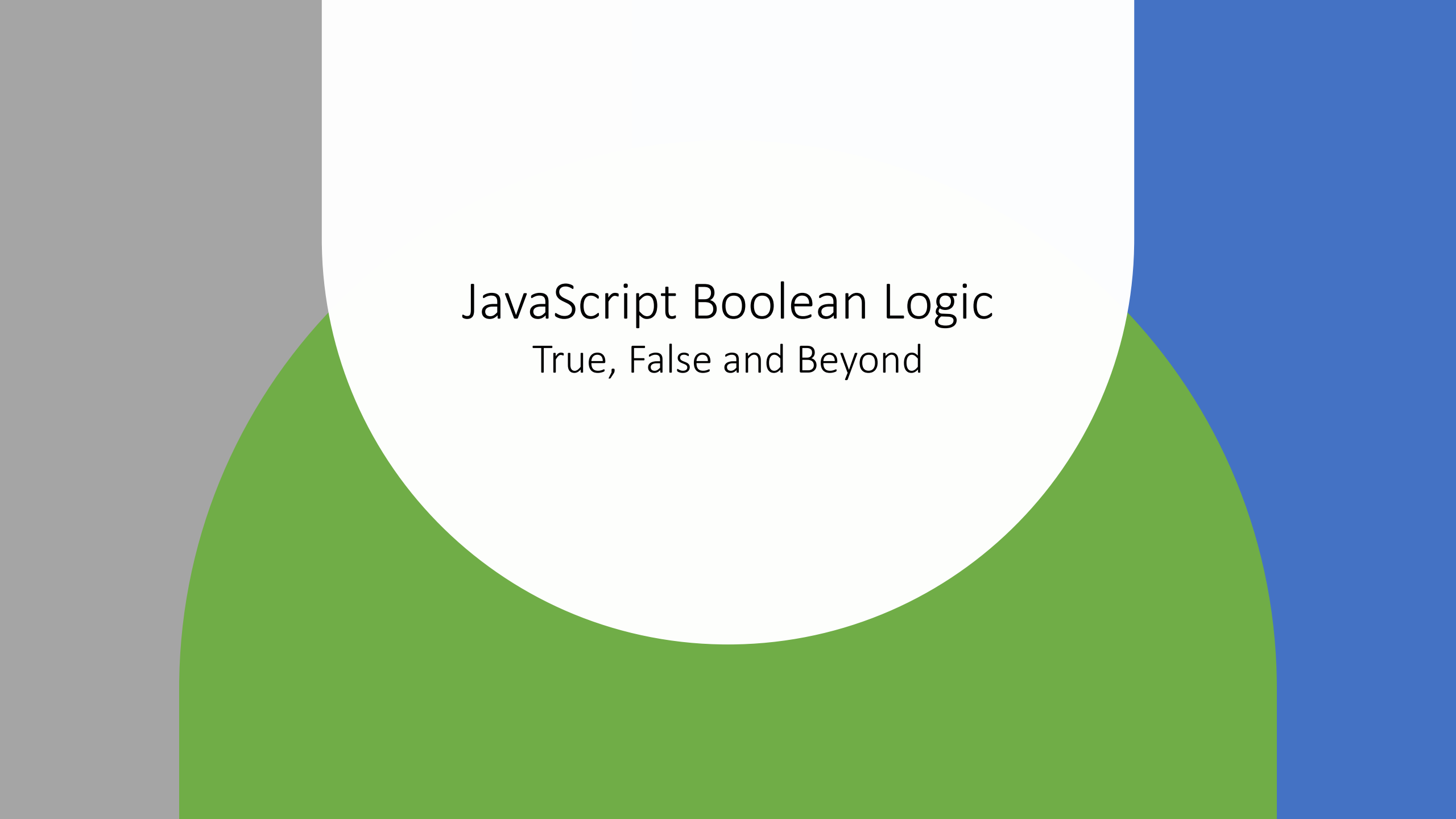
Variables, Strings, Prompt, and Console.log

1. Ask for the user's first name
2. Ask for the user's last name
3. Ask for the user's age
4. Print out the user's full name in a sentence
5. Print out the user's age in a sentence



JS Exercise

Age Calculator



JavaScript Boolean Logic

True, False and Beyond

Comparison Operators

Assuming x = 5

Operator	Name	Example	Result
>	Greater than	x > 10	
>=	Greater than or equal to	X >= 5	
<	Less than	X < -50	
<=	Less than or equal to	X <= 100	
==	Equal to	X == '5'	
!=	Not equal to	x != 'b'	
===	Equal value and type	x === '5'	
!==	Not equal value or equal type	x !== '5'	

Logical Operators

AND

		RESULT
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

Logical Operators

OR

		RESULT
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

Logical Operators

NOT

!(TRUE) = FALSE

!(FALSE) = TRUE

Logical Operators

AND, OR, and NOT

Operator	Name	Example	Result
&&	AND	<code>X < 10 && X !== 5</code>	
 	OR	<code>Y > 9 X === 5</code>	
!	NOT	<code>!(X === Y)</code>	

Assuming `x = 5` AND `y = 9`

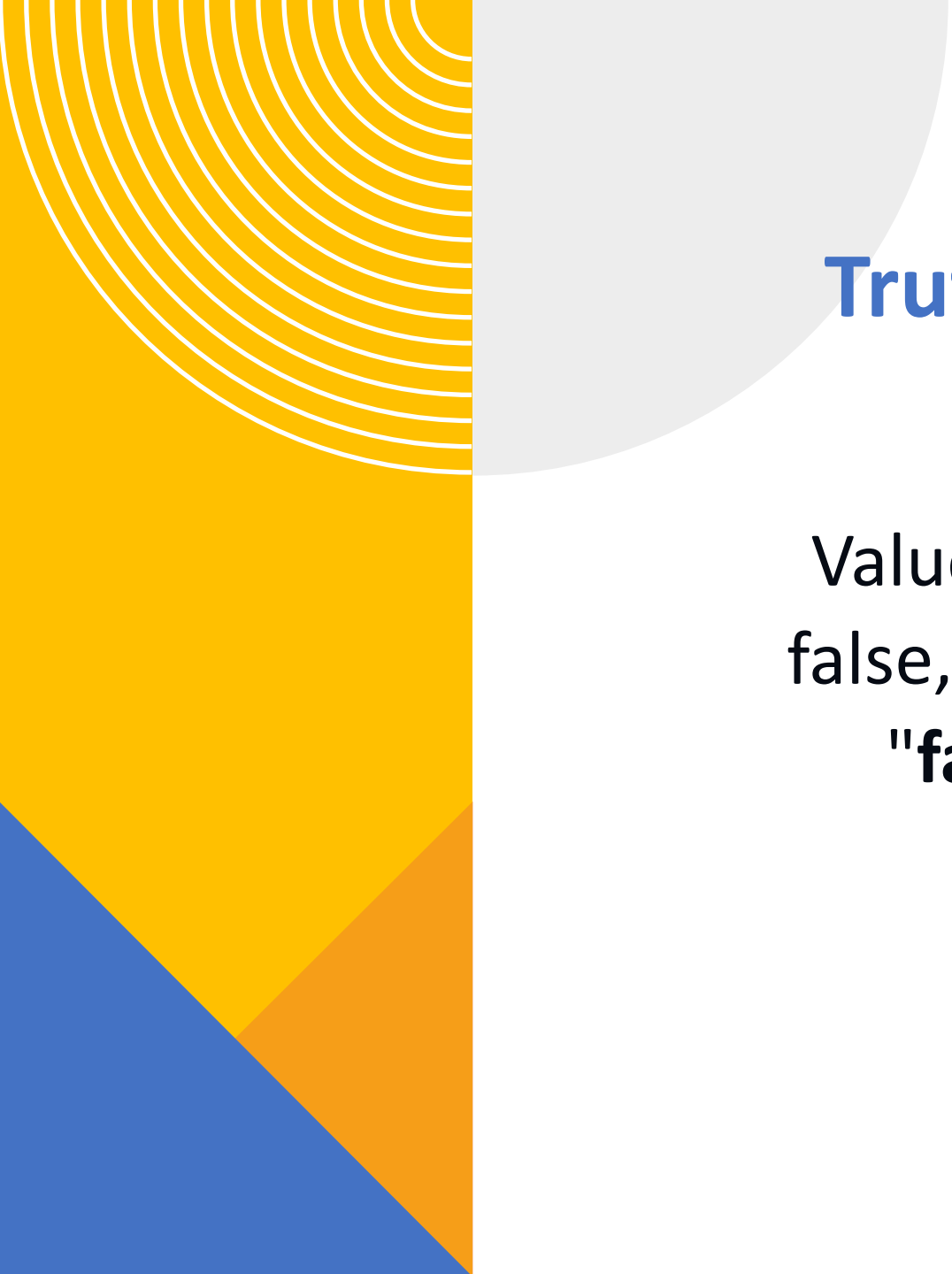
Exercise 1

```
var x = 10;  
var y = "a"
```

```
y === "b" || x >= 10
```


Exercise 2

```
var x = 3;  
var y = 8;  
  
!(x == "3" || x === y) && !(y != 8 && x <= y)
```



Truthy and Falsely Values

Values that aren't actually true or false, are still inherently "**truthy**" or "**falsely**" when evaluated in a Boolean context



TRUTHY AND FALSELY VALUES

- Falsely Values

- False
- 0
- “ ”
- null
- Undefined
- NaN

Exercise 3

```
var str = ""  
var msg = "haha!"  
var isFunny = "false"  
  
!(( str || msg ) && isFunny)
```

JavaScript Variables

JavaScript Variables

In JavaScript, variables are used to store data that can be referenced and manipulated in your code. There are three main ways to declare variables:

- **Var**
- **let**
- **const**



`var` Declaration

Definition:

var is the oldest way to declare a variable in JavaScript. It was used before let and const were .

Scope:

var is function-scoped, meaning it is accessible within the function it is declared in. If declared outside any function, it is globally scoped.

Hoisting:

Variables declared with var are hoisted to the top of their scope. This means you can use the variable before it is declared, though the value will be undefined until the line where it's assigned.

`var` Declaration

```
console.log(fullname);  
var fullname = "John";  
console.log(fullname);
```

Key Points:

It can be re-declared and updated within the same scope.

`let` Declaration

Definition:

let is a modern way to declare variables, introduced in ES6 (2015). It provides block-level scope.

Scope:

let is block-scoped, meaning it is only accessible within the block (e.g., {}) it is declared in.

Hoisting:

let is hoisted, but unlike var, it is not initialized. You cannot use it before the declaration; doing so will result in a ReferenceError.

`let` Declaration

```
console.log(fullname);  
let fullname = "John";  
console.log(fullname);
```

Key Points:

It can be updated within its block scope but cannot be re-declared within the same scope.

`const` Declaration

Definition:

const is also introduced in ES6 (2015) and is used to declare variables whose values should not change (constants).

Scope:

const is block-scoped, similar to let

Hoisting:

const is hoisted, but like let, it is not initialized and cannot be accessed before declaration.

`const` Declaration

```
const name = "Jack";  
console.log(name); // Outputs: "Jack"  
  
// name = "Jill"; // TypeError: Assignment to constant variable.
```

Key Points:

- `const` is used when you want to make sure a variable's value doesn't change after it's been set.
- You must assign a value to a `const` variable when declaring it.

Differences Between var, let, and const

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function-scoped	Block-scoped	Block-scoped
Hoisting	Hoisted (initialized as <code>undefined</code>)	Hoisted (not initialized)	Hoisted (not initialized)
Re-declaration	Allowed	Not allowed in the same scope	Not allowed
Updating Value	Allowed	Allowed	Not allowed (for primitives)
Use Case	Legacy code	When variable values change	When variable values should not change

JavaScript Scope

JavaScript Scope

Scope refers to the **context** or **environment** in which variables are accessible and can be referenced.

Scope determines where variables, functions, and objects are visible and can be used in your code.

There are several types of scope in JavaScript:

- Global Scope
- Function Scope
- Block Scope



Global Scope

Variables declared outside of any function or block are in the global scope. They can be accessed from anywhere in your code.

```
var globalVar = "I am global";

function showGlobalVar() {
  console.log(globalVar); // Accessible here
}

showGlobalVar();
console.log(globalVar); // Also accessible here
```


Function Scope

Variables declared inside a function are function-scoped. They can only be accessed within that function.

```
function showFunctionVar() {  
  
    var functionVar = "I am inside a function";  
  
    console.log(globalVar); // Accessible here  
  
}  
  
showFunctionVar();  
console.log(functionVar); // Also accessible here
```

Block Scope

Variables declared with `let` or `const` inside a block (e.g., within `{}`) are block-scoped. They are only accessible within that block.

```
if (true) {  
  let blockVar = "I am inside a block";  
  
  console.log(blockVar); // Accessible here  
}  
  
console.log(blockVar); // Error: blockVar is not defined
```

JavaScript Function

JavaScript Function

Introduction

Functions are one of the core building blocks in JavaScript. They help us organize code, reuse logic, and build modular programs.

What is a Function?

A **function** is a reusable block of code designed to perform a particular task.



Syntax

```
function functionName() {  
    // code to be executed  
}
```

Example

```
function greet () {  
  
  console.log("Hello, student!");  
}  
  
greet(); // Output: Hello, student!
```

Exercises

1. Create a function called `sayHello` that prints `"Hello World!"`.
2. Create a function called `farewell` that prints `"Goodbye!"`.

Parameters and Arguments

- **Parameters:** Variables listed inside the function parentheses.
- **Arguments:** The actual values passed to the function.

```
function greetUser(name) {  
  
    console.log("Hello, " + name);  
  
}  
  
greetUser("James");
```


Exercises

1. Create a function called **add** that takes two numbers and logs their sum.
2. Create a function called **subtract** that takes two numbers and logs their difference.

Return Values

Return keyword is used to send back a result from a function.

```
function add(x, y) {  
    return x + y;  
}  
  
let result = add(5, 3);  
  
console.log(result);
```

Exercises

1. Create a function that **multiplies** two numbers and returns the result.
2. Create a function called **divide** that divides two numbers and returns the quotient.

Function Expressions

Functions can also be stored in variables.

```
const greet = function(name) {  
  return "Hello, " + name;  
};  
  
console.log(greet("Ada"));
```

Exercises

1. Write a function expression that returns the square of a number.
2. Write a function expression that takes two parameters and returns their product.

Arrow Functions

Shorter syntax for writing functions.

```
const greet = (name) => {  
  return "Hello, " + name;  
};
```

```
const greet = name => "Hello, " + name;
```

Exercises

1. Create an arrow function `double` that returns twice a given number.
2. Create an arrow function `square` that returns the square of a given number.

Default Parameters

Assign default values if no arguments are passed.

```
function greet(name = "Guest") {  
  console.log("Hello, " + name);  
}  
  
greet()
```


Exercises

1. Create a function that greets with a default message if no name is passed.
2. Create a function calculateTotal that takes price and quantity as parameters, with quantity defaulting to 1.

Callback Functions

A function passed into another function as an argument.

```
function processUserInput(callback) {  
  
  let name = "John";  
  
  callback(name);  
  
}  
  
processUserInput(function(name) {  
  
  console.log("Hello, " + name);  
  
});
```

Exercises

isEven()

Write a function *isEven()* which takes a single numeric argument and returns true if the number is even, and false otherwise

```
isEven(4);    //true  
isEven(21);   //false  
isEven(68);   //true  
isEven(333);  //false
```

Exercises

factorial()

write a function `factorial()` which takes a single numeric argument and returns the factorial of that number

4! is $4 \times 3 \times 2 \times 1$

6! is $6 \times 5 \times 4 \times 3 \times 2 \times 1$

0! is 1

```
factorial(5);    //120
factorial(2);    //2
factorial(10);   //3628800
factorial(0);    //1
```

JavaScript Function Exercises

- Check if a number is even.
- Check if a number is odd.
- Find the square of a number.
- Find the cube of a number.
- Add two numbers.
- Find the larger of two numbers.
- Check if a number is positive or negative.
- Find the factorial of a number.
- Return the first character of a string.
- Check if a number is divisible by 5.
- Multiply two numbers.
- Find the absolute value of a number.
- Swap two numbers.
- Find the length of a string.
- Find the sum of all numbers from 1 to n.
- Count vowels in a string.
- Check if a string is empty.

JavaScript Arrays

Javascript Arrays

Our First Data Structure

JavaScript Arrays

What is an Array?

An array is a data structure that holds a list of items (elements) in a single variable. Each item is identified by an index, starting from 0.

```
let fruits = ["apple", "banana", "orange"];
```


Suppose I wanted to model a group of friends:

```
var friend1 = "Charlie";  
var friend2 = "Liz";  
var friend3 = "David";  
var friend4 = "Mattias";
```

This is a lot of code, and it doesn't let us group the friends together

This is a perfect use case for an ARRAY

```
var friends = ["Charlie", "Liz", "David", "Mattias"];
```

Arrays

Arrays let us group data together in lists

```
var friends = ["Charlie", "Liz", "David", "Mattias"];
```



Array are indexed starting at 0. Every slot has a corresponding number

Accessing Elements in an Array

You can access elements using bracket notation with the index.

```
let colors = ["red", "green", "blue"];
```

```
console.log(colors[1]);
```

Arrays

We can use those indices to retrieve data



```
var friends = ["Charlie", "Liz", "David", "Mattias"];  
console.log(friends[0])    //"Charlie"  
friends[1] + " <3 " + friends[2]    //"Liz <3 David"
```

Exercises

1. Create an array of animals and log the first and last elements.
2. Access and log the third element in an array of at least five items.

Modifying Elements

You can change the value of elements by assigning a new value to a specific index.

```
let numbers = [1, 2, 3];
```

```
numbers[1] = 20;
```

Arrays

We can also update arrays



```
var friends = ["Charlie", "Liz", "David", "Mattias"];  
  
friends[0] = "Chuck";  
friends[1] = "Lizzie";
```



Exercises

1. Replace the second element in an array of three cities with another city.
2. Change the value of the last item in an array of five numbers.



Array Methods: push(), pop()

- The push() method of Array instances adds the specified elements to the end of an array and returns the new length of the array.
- The pop() method of Array instances removes the last element from an array and returns that element. This method changes the length of the array.



```
let books = ["Book A", "Book B"];
```

```
books.push("Book C");
```

```
books.pop();
```

Exercises

1. Use `push()` to add two new elements to an empty array.
2. Use `pop()` to remove the last element from an array of four fruits.



Array Methods: shift(), unshift()

- The shift() method of Array instances removes the first element from an array and returns that removed element. This method changes the length of the array.
- The unshift() method of Array instances adds the specified elements to the beginning of an array and returns the new length of the array.



```
let queue = [2, 3, 4];
```

```
queue.shift();
```

```
queue.unshift(1);
```

Exercises

1. Use `shift()` to remove the first item from an array of numbers.
2. Use `unshift()` to add a new color to the beginning of an array.
3. Write a for loop to print each item in an array of car brands.
4. Use `forEach()` to log all elements in an array of movie titles.

Array Length

- Use `.length` to find how many items are in an array.

```
let arr = [10, 20, 30];  
  
console.log(arr.length);
```

Looping Through Arrays

- The `forEach()` method of Array instances executes a provided function once for each array element.

```
for (let i = 0; i < fruits.length; i++) {  
  
    console.log(fruits[i]);  
  
}  
  
fruits.forEach(function(fruit) {  
  
    console.log(fruit);  
  
});
```


Exercises

1. Write a for loop to print each item in an array of car brands.
2. Use `forEach()` to log all elements in an array of movie titles.

Todo List

- "new" - Add A Todo
- "list" - View all todos
- "quit" - Quit App